

JAVASCRIPT

Cours suivi avec Guillaume P. (<https://github.com/Guillaume-Pollastri>) sur la plateforme ;
<https://www.boot.dev/>

Plusieurs objectif avec ce document :

- *découvrir JavaScript avant de toucher à TypeScript (un super set de Javascript)
- *commencer à se préparer pour ft_transcendance qui a dans sa stack techno JS/typescript
- *à ouvrir plus de porte grâce à des compétences sur JS / Typescript qui peut être utiliser pour du back end qu'on voit dans différentes offres d'emplois*.

Ce document par du principe que vous avez des bases dans un autre langage de programmation (C / C++/python) et que certaines choses n'ont pas à être expliquer.

Ce document reste un « guide de survie », il touche des bases mais ne se veut pas être parfait ou complet sur l'ensemble du langage. Ce document est le résultat de prise de note d'un cours, si vous souhaitez tout apprendre parfaitement ce document n'est pas le bon.

SOMMAIRE

[Histoire](#)

[Compilation, Interprétation, JIT,.....](#)

[DOM](#)

[BOM](#)

[Print](#)

[Les variables](#)

[les types](#)

[var, let et const](#)

[la mémoire](#)

[typeof](#)

[if et switch](#)

[le point-virgule](#)

[les opérateurs logiques et tests \(les problèmes\)](#)

[opérateur ??](#)

[=== vs ==](#)

[truthy and falsy](#)

[fonctions](#)

[fonctions anonymes](#)

[« arrow functions »](#)

[return value](#)

[IIFE](#)

[« Objets »](#)

[déclaration](#)

[const mais pas tellement](#)

[accès](#)

[delete](#)

[Pass by reference](#)

[Spread Syntax](#)

[Classes](#)

[constructor](#)

[get et set](#)

[Heritage et Super\(\)](#)

[For loop](#)

[Array, set et map](#)

[Array](#)

[Set](#)

[Map](#)

[Promises et Event Loop](#)

[Promesse the old way](#)

[The new way](#)

[Behind the magic](#)

[Le REPO de la honte](#)

Histoire

Javascript est un langage de programmation qui a été créé par Netscape (un navigateur web dans les années 90) en 1995 en 10 jours par un ingénieur de l'entreprise.

L'idée était de rendre dynamiques les pages internet qui était alors en 1994 encore statique, avec un langage de programmation proche de JAVA (qui était très populaire), mais plus accessible.

Le choix du nom est totalement marketing, proche de JAVA qui était la star des années 90 / bulle internet.

Netscape va perdre la guerre contre Internet Explorer mais Internet Explorer va bien utiliser Javascript.

Depuis 2009 et l'arrivée de Node.js, l'exécution de javascript peut se faire en dehors du navigateur. Javascript est donc utilisé un peu partout maintenant en dehors de navigateur pour du back end et autres.

En 2012, sort Typescript qui est un super set de Javascript. C'est-à-dire qu'il vient ajouter des choses aux langages. Tout code JS est du Typescript valide mais aucun Typescript n'est du JS valide. Il vient ajouter de la sécurité sur les types en gros à JS (à voir dans un autre document sur Typescript).

Fun fact ; Javascript est une marque déposée qui appartient à Oracle (ancien Sun Microsystems, qui avait collaboré avec netscape). Techniquement on ne peut pas l'utiliser comme on veut.

ECMAScript est le standard pour Javascript.

Compilation, interprétation, JIT, etc.....

JavaScript est historiquement un langage de programmation interprété. C'est-à-dire qu'il est lu ligne par ligne et exécute ce qu'il voit. A contrario du C et C++ qui sont compilés en premier lieu pour fournir un exécutable.

Cependant, il y a eu des changements avec JS moderne. On a vu l'apparition d'un JIT.
Just In Time compiler.

Ça va permettre de compiler votre code javascript à la volée et d'avoir de meilleures performances pour l'interpréteur (attention aucun rapport avec votre code qui est toujours lent pour cause de skills issues).

DOM

= Document Object Model

C'est une interface de programmation pour le HTML et CSS des pages web. Ça représente notre page web et ça peut être modifiée par notre code en Javascript.

BOM

= Browser Object Model

C'est ce qui nous permet d'interagir avec le navigateur web en particulier.

PRINT

Dans votre code pour print sur STDOUT ;

```
console.log("<some text>");
```

console.log() => c'est que qui permet de print

"<some text>" => le texte doit être entre délimiter de string (voir plus tard dans les types). On peut le remplacer par des variables.

A noter que console.log() se comporte comme printf (ou print de python3) si on veut print des chaines de caractères avec des variables ;

```
const shadeOfRed = 101;
```

```
console.log(` The shade is ${shadeOfRed}`);
```

Sur la première ligne on définit une variable (à voir après).

Dans la seconde ligne on utilise \${ } pour print la valeur de la variable.

A noter qu'on utilise les backquotes ici

On peut aussi faire ;

```
console.log("Sent is null:", sentMessages === null);
```

cela va print «Sent is null : true »

C'est un comportement très proche du printf() en C.

LES VARIABLES

Les types

En terme de type primitifs on a ;

*boolean => true, false

*string => chaîne de caractère

*number => qui représente les int et les floats

*undefined => une variable à qui on n'a pas assigné quoi que ça soit

*null => une variable à qui on a mis null et qui peut contenir tout type de valeur

*BigInt => un type qui peut contenir un «big int » 2^{53} , il faut que cela termine par un n

Var, let, const

Pour la déclaration de variable ;

var <nom de variable>

Par défaut la valeur de cette variable est **undefined**. Toute variable déclare mais non initialise a une valeur undefined.

L'utilisation de var pour la déclaration de variable permet plusieurs choses.

On peut redéclarer la variable plusieurs fois sans que ça ne crée de problème de compilation ou autres (ce qui pourrait en réalité être un problème qui crée une erreur plus tard dans le code).

L'autre point important de var, c'est son scope qui est plus large que les autres mots clé nécessaire pour déclarer une variable.

Les 2 autres mots pour déclarer une variable sont ; let et const

let <nom de la variable>

const <nom de la variable>

différences de scope

```
var x;
console.log("value of x =", x);

//new block scope
{
    console.log("value of x =", x);
    let y = 42;
    console.log("value of y =", y);
    const z = 43;
    console.log("value of z =", z);
}

//outside block scope
console.log("value of y =", y = undefined); // the "= undefined" allow to avoid errors
console.log("value of z =", z = undefined); //will take only the value undefined IF it's not existing

//new block scope
{
    var other = "les chats";
    console.log("value of other =", other);
}

//outside block scope
console.log("value of other =", other);
```

```
~/JS$ nodejs main.js
value of x = undefined
value of x = undefined
value of y = 42
value of z = 43
value of y = undefined
value of z = undefined
value of other = les chats
value of other = les chats
```

On voit ici que var n'est pas limitée ici par un block scope mais qu'il est plus large. X et other peuvent être utilisés ailleurs que leur scope de base tandis que y et z sont plus limités.

Le scope d'une fonction, qu'on verra plus tard, par contre prime et « enclose » bien var aussi.

Différences entre let et const

Var peut être redéclarer ;

```
//code valide
var x = 42;
var x = 0;
console.log(x);
```

Mais let et const ne peuvent pas être redéclarer ;

```
//code valide
var x = 42;
var x = 0;
console.log(x);

//code invalide
let y = "les chats";
let y = "les chiens";
~
```

```
let y = "les chiens";
^

SyntaxError: Identifier 'y' has already been declared
    at internalCompileFunction (node:internal/vm:73:18)
    at wrapSafe (node:internal/modules/cjs/loader:1274:20)
    at Module._compile (node:internal/modules/cjs/loader:1320:27)
    at Module._extensions..js (node:internal/modules/cjs/loader:141:10)
    at Module.load (node:internal/modules/cjs/loader:1197:32)
    at Module._load (node:internal/modules/cjs/loader:1013:12)
    at Function.executeUserEntryPoint [as runMain] (node:internal/main:128:12)
    at node:internal/main/run_main_module:28:49
```

Ensuite la différence entre let et const, est le fait que const est....constant !

```
//code valide
var x = 42;
var x = 0;
console.log(x);

//code valide
let y = "les chats";
y = "les chiens";

//code invalide
const value = true;
value = false;
```

```
value = false;
      ^
```

```
TypeError: Assignment to constant variable.
```

La mémoire

Les types primitifs qu'on a pu décrire sont sur la stack, tandis que les objets eux vont être mis sur la heap. On a pas à gérer particulièrement tout ça, comme en python car c'est géré pour nous.

Typeof

Un outil qui peut se trouver utile en javascript est typeof, qui va nous donner le type de la variable.

Comme vous avez pu le voir en JS, on est pas sur du type strict (d'où typescript).

```
let value = "les chats";
console.log("le type de la variable = ", typeof value);

value = 42;
console.log("le type de la variable = ", typeof value);

value = true;
console.log("le type de la variable = ", typeof value);

value = 93759205723095729572309457230945735478098547n;
console.log("le type de la variable = ", typeof value);

value = undefined;
console.log("le type de la variable = ", typeof value);
```



```
:~/JS$ nodejs main.js  
le type de la variable = string  
le type de la variable = number  
le type de la variable = boolean  
le type de la variable = bigint  
le type de la variable = undefined
```

IF et SWITCH

Très proche de la logique de C ou d'autres langages. Voici des structures d'exemples pour illustrer la syntax;

```
let x = 42;

if (x < 42) {
    console.log("< 42");
} else if (x > 1317) {
    console.log("leet");
} else {
    console.log("other");
}
```

Pour le switch, ce qui est intéressant compare a d'autres langages c'est la possibilité d'utiliser des string pour comparer (on pense au C++);

```
let x = "les chats";

switch (x) {
    case "les chiens":
        console.log("les chiens");
        break;
    case "les serpents":
        console.log("les serpents");
        break;
    case "les chevalSSSSSS":
        console.log("les cheval$$$$$");
    default:
        console.log("les chats");
        break;
}
```

Le point-virgule

Depuis le début de ce document, vous avez pu voir que je mets un point-virgule à chaque fin de ligne.

Mais il faut savoir qu'il n'est pas obligatoire !

```
let x = "les chats"
let y = "les serpents"
console.log(x, y)
```

Cependant, il reste conseiller de le mettre.

Il existe des cas où cela reste obligatoire comme ici ;

```
let x = "les chats", y //valide avec y qui est undefi
let variable = "les chiens" ; let nb = 42n;
console.log(x, y)
console.log(variable, nb)
```



les opérateurs logiques et tests (les problèmes)

Les opérateurs logiques / de comparaisons sont très semblables à ceux qu'on connaît en C / C++.

Donc on retrouve ;

&& and

|| or

! not

Mais aussi les classiques ;

>=

<=

<

>

Jusque-là tout va bien ! Maintenant des nouveautés et des problèmes

Opérateur ??

L'opérateur ?? est un outil très sympa qui va fonctionner de cette manière ;

<lhs evaluation> ?? <rhs evaluation>

lhs et rhs pour "left hand sign" et "right hand sign".

Si une des deux expressions est undefined ou null, il va utiliser l'autre ;

```
let x = null;
let y = "les chats"
let z; //undefined

console.log(x ?? y);
console.log(z ?? y);
console.log(y ?? z);
```

Output ;

```
les chats
les chats
les chats
```

=== VS ==

Un des problèmes de Javascript est un comportement qui peut vite arriver avec un oubli ou une faute de frappe.

Il existe un opérateur === et l'autre classique ==

```
let a = "42";
let b = 42;

console.log(a == b);
console.log(b == a);

console.log("test with === ");
console.log(a === b);
console.log(b === a);
```

Output ;

```
true
true
test with ===
false
false
```

Qu'est ce qui se passe ?

Quand on utilise le == il va regarder si c'est égal en faisant une conversion de type.
Alors que le === ne fait pas cette conversion de type.

Bien sûr on voit des applications pratiques à ça (et j'imagine que pour le front end ça doit être encore bien) mais c'est aussi la cause de pas mal de problèmes et de bugs difficiles à détecter dans un grand projet.

Donc par défaut, on va utiliser plus souvent le === et utiliser le == uniquement si on en a vraiment le besoin.

Truthy and Falsy

Par truthy and falsy, qui sont des termes inventés, on entend des choses qui déclenchent true ou false.

On a le grand classique ;

```
if (0) {
    console.log("0");
}
if (1) {
    console.log("1");
}
if (-1) {
    console.log("-1");
}
```

Ici il y a que 1 et -1 qui déclenchent le console.log().

Mais il existe une liste d'autres choses qui peuvent déclencher des true ou false (dont certains sont étonnants)

Liste de déclenchement de true:

- true
- autre valeur que 0
- non-empty string
- [] (array vide)
- {} (objet vide)
- function() {} (fonction vide)
- Infinity => si on fait des divisions par 0

Liste de déclenchement de false :

- false
- 0
- "" (string vide) (le cas le plus bizarre)
- null
- undefined
- NaN (Not a Number) => on peut l'obtenir avec 0/0

Fonctions

Pour déclarer une fonction on a besoin d'utiliser le mot clé ; fonction puis le nom et les arguments.

```
function sum(a, b) {  
    return a + b;  
}  
  
console.log(sum(4,2));  
console.log(sum("les ", "chats"));
```

Une précision assez intéressante qui est opposée à d'autres langages de programmation. C'est qu'on peut appeler la fonction avant même d'avoir fait sa définition ;

```
//code valide qui fonctionne  
console.log(sum(4,2));  
console.log(sum("les ", "chats"));  
  
function sum(a, b) {  
    return a + b;  
}
```

Compare à python => on ne peut pas faire plusieurs valeurs de retour, c'est limité à une valeur return.

Fonctions anonymes

On peut créer des fonctions anonymes. C'est-à-dire une fonction qui n'a pas de nom.

La structure à respecter ;

```
function (<parameter>){  
    <code>;  
}
```

```
// using an anonymous function  
conversions(  
    function (a) {  
        return a + a;  
    },  
    1,  
    2,  
    3,  
);  
// 2 4 6
```

« Arrow functions »

Une arrow function, permet d'écrire différemment et de manière plus légère une fonction.

```
// using the fat arrow syntax
const add = (x, y) => {
  return x + y;
};
```

Cependant cela a des implications importantes sur certains comportements avec this, arguments et super. Car ils n'ont pas le même scope

Le « this » dans les arrow function, n'est pas celui de l'objet si on l'utilise en tant que méthode.

Mais le this du «scope parent »

(même si on a pas encore parler des objets dans cette documentation, je vous invite à regarder la prochaine partie. Cette partie étant relative aux fonctions je me dois de le mettre ici).

```
let name = "le";
let surname = "chat";

const obj = {
  name: "les",
  surname: "chiens",
  test: () => {console.log(this.name, this.surname)},
  test2() {console.log(this.name, this.surname)},
}

obj.test();
obj.test2();
```

Output ;

```
undefined undefined
les chiens
```

Voici aussi un exemple si on met pas de this ;

```
var test = "chats";

obj = {
  test: "chien",
  my_func: () => {console.log(test)},
}

obj.my_func();//chats
```

Return value

A contrario de python, et comme certains autres langage de programmation. On ne peut pas return plusieurs valeurs avec une fonction.

Mais bien sur il y a des moyens de manipuler cela avec plusieurs notions qu'on va voir par la suite. Je vais quand même le montrer ici car c'est plus logique et vous pourrez revenir dessus plus tard.

Méthode 1 ; utiliser un objet

```
function multiple_return() {  
  let x = 4;  
  let y = 2;  
  return {x, y};  
}  
  
let {x, y} = multiple_return();  
  
console.log(x);  
console.log(y);
```

return d'un objet

on déclare un x et y qui vont prendre la valeur 4 et 2

X = 4, y = 2 et si on print bien leurs typeof on aura bien des number.

Méthode 2 ; utiliser un array

```
function multiple_return() {  
  let x = 4;  
  let y = 2;  
  return [x, y];  
}  
  
let [x, y] = multiple_return();  
  
console.log(x);  
console.log(y);
```

return un array avec nos deux valeurs

on a bien un array avec x et y

Dans tous les cas on parle de « *destructuring* »

IIFE

IIFE = immediate Invoked Function Expression

Il faut savoir qu'on peut enregistrer une fonction dans une variable et utiliser cette variable pour appeler la fonction, un peu à la manière dont j'ai illustré la arrow function.

```
let x = function(a,b) {console.log(a + b)};  
x(2,6); //print 8
```

Mais on peut aussi créer une fonction qui va être invoquée immédiatement.

Et on n'est pas obligé de le mettre dans une variable.

```
{function (a,b) {console.log(a + b); return 1;}} (2,6);
```

A noter que j'ai dû mettre des parenthèses autour de la fonction. Celle-ci va tout de suite être exécutée avec la valeur 2 et 6.

« OBJETS »

Pourquoi objets entre guillemets ? Tout simplement parce qu'il peut y avoir une différence de terme avec d'autres langages de programmation.

Dans certains langages de programmation, la notion d'objet veut dire que c'est une instance d'une classe. Or ici, ce n'est pas le cas ! C'est quelque chose d'appart qui est propre à JS.

Pour le représenter, je me dis que c'est initialement parce qu'on a javascript qui est utilisé pour le front end du web. Les pages HTML qu'on veut rendre dynamique ce sont des objets comme on va le voir. Bien sûr les objets ce n'est pas que ça, c'est plus un moyen de se représenter la chose.

Déclaration

```
<déclaration de variable> = {  
  <nom_attribut> : <value_attribut>,  
  <nom_attribut> : <value_attribut>,  
  <nom_fonction>() { <ce que fait la fonction> },  
}
```

```
const obj = {  
  name: "les",  
  surname: "chats",  
  myFunction() {console.log(this.name, this.surname)},  
}  
  
obj.myFunction();
```

Ici il faut noter l'utilisation du mot this. Qui sert comme en C++ (mais qui n'est pas obligatoire en C++) de préciser qu'on parle du name et surname de l'objet.

Attention sur les fonctions à aller voir la partie précédente sur les arrow functions. Noter aussi qu'il n'y a pas besoin de mettre le mot clé « function » devant celle-ci.

Const mais pas tellement non plus

On va souvent avoir tendance à mettre les objets en const. Cependant autant pour une variable on ne peut pas changer la valeur ou la redéclarer. Par contre sur les objets on peut faire des modifications ;

```
const test = "yes";
//test = "nope"; pas possible ici car const

const obj = {
  name: "les",
  surname: "chats",
  myFunction() {console.log(this.name, this.surname)},
}

obj.myFunction();// ici output = les chats

obj.name = "los";
obj.surname = "gatos";
obj.myFunction();//ici output = los gatos
```

En fait c'est l'intérieur des objets qu'on peut modifier.

On peut même aller plus loin dans l'idée et ajouter des champs a notre objet même après création.

```
const obj = {
  name: "les",
  surname: "chats",
  myFunction() {console.log(this.name, this.surname)},
}

obj.nouveau = "serpents";//ajout d'un nouveau champs

console.log(obj.nouveau);//output serpent
```

Accès

Comme on a pu le voir dans le précédent point pour accéder a un champs il faut utiliser le .

<objet>.<attribut>

ou pour une fonction

<objet>.<nom de la fonction>()

Cependant le dernier point, celui d'ajouter un champs a la vole, vient poser un problème. Est-ce que le champs que je veux atteindre existe ou non ?

Il existe une possibilité avec le point d'interrogation

<objet> ?.<attribut>

```
const obj = {
  name: "les",
  surname: "chats",
  myFunction() {console.log(this.name, this.surname)},
}

obj.nouveau = "serpents";//ajout d'un nouveau champs

console.log(obj?.nouveau);//output serpent
console.log(obj?.cat);//undefined car n'existe pas
```

Cela permet d'assurer une undefined ou null et peut permettre d'éviter une erreur (même si normalement sur nodejs vous n'aurez pas d'erreur => mais still implementation defined et other shit).

Cet opérateur est le « **optional chaining operator** »

Il y a un autre moyen d'accéder.

<nom de l'objet>[«nom du champs»]

```
const test = "yes";
//test = "nope"; pas possible ici car const

const obj = {
  name: "les",
  surname: "chats",
  myFunction() {console.log(this.name, this.surname)},
}

obj.nouveau = "serpents";//ajout d'un nouveau champs

console.log(obj["surname"]);//output => chats
```

Delete

On peut bien sur delete un élément sans problème

delete <objet>.<champs>

```
const test = "yes";
//test = "nope"; pas possible ici car const

const obj = {
  name: "les",
  surname: "chats",
  myFunction() {console.log(this.name, this.surname)},
}

console.log(obj);//output { name: 'les', surname: 'chats', myFunction: [Function: myFunction] }

delete obj.name;

console.log(obj);//{ surname: 'chats', myFunction: [Function: myFunction] }
```

Pass by reference

On ne peut pas passer par référence sur les types primitifs ! Mais on peut sur les objets !
En fait cela est dû au fait que les objets vivent sur la heap tandis que les types primitifs vivent sur la stack.

```
function plus(x) {
    x++;
}

function plus_obj(obj) {
    obj.x++;
}

let x = 41;
plus(x);
console.log(x); //output 41

let obj = {x:41};
plus_obj(obj);
console.log(obj.x); //output 42

//other interesting test
plus(obj.x);
console.log(obj.x); //output 42
```

On peut aussi avoir ce cas

```
const obj1 = {x:4, y:2};
const obj2 = obj1;

obj2.x++;
console.log(obj1.x); //print 5
```

Spread syntax

On peut ajouter des objets ensemble mais pour cela il faut suivre la logique de « spread syntax »

```
obj1 = {test: "les chats"};
obj2 = {test2: "les chiens"};

obj3 = obj1 + obj2;

console.log(obj3.test); //undefined
console.log(obj3.test2); //undefined

obj4 = {...obj1, ...obj2}; //spread syntax

console.log(obj4.test); //les chats
console.log(obj4.test2); //les chiens
```

Les 3 . qui sont devant sont nécessaires pour que cela prenne les champs.

Attention lorsque l'on va faire cela avec des champs identiques, cela reste une shallow copy et va prendre l'élément de droite ;

```
obj1 = {test: "les chats"}; //attention test meme nom  
obj2 = {test: "les chiens"}; //attention test meme nom  
  
obj4 = {...obj1, ...obj2}; //spread syntax  
  
console.log(obj4.test); //les chiens
```

C'est une syntaxe qu'on va retrouver dans les arrays.

Classes

Ajout récent de javascript (2015), elle se comportent comme on peut l'attendre des classes dans d'autres langages mais avec des petits détails qu'on va voir ici.

Constructor

```
class MyClass {
  constructor(value1, value2) {
    this._value1 = value1;
    this._value2 = value2;
  }
}

let instance = new MyClass(4,2);
console.log(instance._value1, instance._value2);
```

Ceci est un exemple de déclaration de classe. On utilise le keyword **class** et **constructor**. Constructor prend différents arguments en fonction de ce que l'on souhaite avoir dans notre objet. A noter que pour instancier un objet on va utiliser le keyword **new** (mais pas besoin de faire de delete comme en C++).

Point important, par défaut comme en python, les attributs sont tous publics. C'est pour ça que j'ai pu imprimer les valeurs depuis console.log().

On peut mettre en privé avec le symbole # mais cela implique une déclaration à faire avant le constructor (sinon il va envoyer une erreur de syntaxe ; *Private field '#_value1' must be declared in an enclosing class*).

```
class MyClass {
  #_value1; //declaration
  #_value2; //declaration
  constructor(value1, value2) {
    this.#_value1 = value1; //utilisation du #
    this.#_value2 = value2;
  }
}

let instance = new MyClass(4,2);
console.log(instance.#_value1, instance.#_value2); //va mettre une erreur de syntaxe
```

Le symbole «_» devant n'est pas obligatoire c'est juste ma manière de faire qui ressemble à du python (mais n'a pas le même comportement que python avec __).

GET et SET

Si on a des attributs privés, on a l'habitude en C++ de faire des setters et des getters, ici en javascript cela est aussi possible mais avec des keywords ; **set** et **get**

L'avantage / différence entre simplement faire une fonction qui return la valeur que l'on cherche c'est la manière dont on appelle set et get.


```

class MyClass {
  #_value1; //declaration
  #_value2; //declaration
  constructor(value1, value2) {
    this.#_value1 = value1; //utilisation du #
    this.#_value2 = value2;
  }

  get GetterValue1() {
    return this.#_value1;
  }

  set SetterValue1(new_value) {
    this.#_value1 = new_value;
  }
}

let instance = new MyClass(4,2);
console.log(instance.GetterValue1); //print 4 et pas besoin de mettre ()
instance.SetterValue1 = 6; //pas besoin de faire .SetterValue1(6);
console.log(instance.GetterValue1); //print 6

```

Heritage et super()

Si on met en place les classes on peut mettre en place l'héritage. On utilise ici le keyword **extends**.

```

class Parent {
  constructor() {
    this.value_x = 21;
  }
}

class MyClass extends Parent { //on precise ici avec extends la classe qu'on souhaite
  #_value1;
  #_value2;
  constructor(value1, value2) {
    super(); //
    this.#_value1 = value1;
    this.#_value2 = value2;
  }

  get GetterValue1() {
    return this.#_value1;
  }

  set SetterValue1(new_value) {
    this.#_value1 = new_value;
  }
}

let instance = new MyClass(4,2);
console.log(instance.value_x);

```

Ici on va bien print la valeur 21 qui est dans value_x.

A noter que dans mon constructor, j'ai dû utiliser `super()` qui va appeler le constructor de l'objet parent. A l'image de python qui utilise aussi le même mot clé, cela permet de faire appelle a la fonction d'un parent.

```

class Parent {
  constructor() {
    this.value_x = 21; //je laisse en plublic
  }
  func_parent() {console.log("les chats sont mignons");};
}

class MyClass extends Parent //on precise ici avec extends la classe qu'on souhaite
  #_value1;
  #_value2;
  constructor(value1, value2) {
    super();
    this.#_value1 = value1;
    this.#_value2 = value2;
  }

  get GetterValue1() {
    return this.#_value1;
  }

  set SetterValue1(new_value) {
    this.#_value1 = new_value;
  }
  heritage() {
    super.func_parent();
  }
}

let instance = new MyClass(4,2);

instance.heritage(); //va print ; "les chats sont mignons"

```

ici j'appelle dans la fonction héritage => la fonction func_parent.

FOR loop

Les boucles for sont basiquement identique à d'autres langages ;

for (let i = 0 ; i < 10 ; i++)

Mais il y a des petites différences avec python ou C++.

Dans un objet si on veut itérer ;

```
const obj = {
  text: "chats",
  num: 42,
}

for (i in obj) {
  console.log(i); //print => text et num
  console.log(obj[i]); //print => chats et 42
}
```

On va utiliser un **for in**. La variable i qu'on a pas besoin de déclarer comparer au for classique, va prendre la valeur du champs. Il faut utiliser l'accès avec [] pour avoir la valeur.

Il existe aussi un autre type de for ; **for of**

mais celui-ci n'est utilisable que dans un objet itérable donc pas un objet mais par exemple un array (voir prochaine section sur les array).

```
const array = ['chats', 'chien', 'serpent']; //declaration array

for (i in array){
  console.log(array[i]); //accès par [ ]
}

for (i of array){
  console.log(i); //accès par i
}
```

On voit ici que la différence est sur l'accès avec i et les []

Array, set et map

Array

Les array sont très proches du fonctionnement des array dans d'autres langages.

On les déclare avec des []

Mais la grosse différence, c'est qu'en javascript on peut avoir tout type de donnée dans un array.

```
const array = ['chats', 'chien', 'serpent']; //valide
const array2 = [42, 'les chats', undefined]; //valide
```

Pour les accès, comme vu dans les boucles for, il suffit de mettre `<array_name>[<index>]`

Comme en C++ ou python il existe des méthodes sur les objets déjà faites ;

Array : méthode slice()

✓ Baseline Widely available

La méthode `slice()` des instances `Array` retourne une [copie superficielle](#) d'une portion d'un tableau sous la forme d'un nouvel objet tableau sélectionné de `start` à `end` (`end` non incluse), où `start` et `end` représentent l'indice des éléments dans ce tableau. Le tableau d'origine ne sera pas modifié.

Exemple interactif

Démonstration JavaScript : Array.prototype.slice()

```
1 const animals = ["ant", "bison", "camel", "duck", "elephant"];
2
3 console.log(animals.slice(2));
4 // Résultat attendu : Array ["camel", "duck", "elephant"]
```

Array : propriété length

✓ Baseline Widely available

La propriété de données `length` d'une instance de `Array` représente le nombre d'éléments dans ce tableau. Sa valeur est un entier non signé sur 32 bits qui est toujours numériquement supérieur au plus grand indice du tableau.

Exemple interactif

Démonstration JavaScript : Array.length

```
1 const clothing = ["chaussures", "chemises", "chaussettes", "pulls"];
2
3 console.log(clothing.length);
4 // Résultat attendu : 4
```

Comme pour les instances de classe, pour créer un tableau vide on doit utiliser le mot clé new

```
const array = new Array();
//const array = []; //pas possible

//ajouter des elements
array.push("chats");
console.log(array); //print ['chats']
```

Sur les array on va aussi retrouver la spread syntax avec les 3 points.

Cette idée d'utiliser `new <objet>` va se retrouver dans tous les objets qui existent déjà par défaut ou ceux qu'on pourrait être amené à importer dans des modules.

Set

Le set est une structure de donnée, qui permet d'avoir une seule occurrence d'un élément (pas de doublon).

```
const myset = new Set();

myset.add("les chats");
myset.add("les chiens");
myset.add(42);

console.log(myset); //output ; { 'les chats', 'les chiens', 42 }
myset.add("les chats"); //ne va rien ajouter
console.log(myset); //output ; { 'les chats', 'les chiens', 42 }
```

Très pratique potentiellement pour tester si on a des doublons ou pour les éliminer

Maps

Les maps sont une structure de donnée qui fonctionnent comme un dictionnaire avec des paires keys - value.

```
const myMap = new Map();

myMap.set("key", "value");
myMap.set("les", "chats");
myMap.set("les", "chiens");

console.log(myMap); //Map(2) { 'key' => 'value', 'les' => 'chiens' }

Map.set("key", "value"); //ne va pas ajouter
console.log(myMap); //Map(2) { 'key' => 'value', 'les' => 'chiens' }

myMap.set("key", "other value"); //va update
console.log(myMap); //Map(2) { 'key' => 'other value', 'les' => 'chiens' }
```

La map présente le même avantage que les sets sur les doublons.

Promises et Event Loop

Promises est prononcé comme « promesses » en français.

Il faut rappeler que Javascript est « single threaded » (un seul thread). Mais il arrive à opérer plein d'opérations dans le navigateur parce qu'il a la capacité de gérer les tâches de manière asynchrone.

On connaît le code qui est synchrone, c'est le plus simple, c'est quand on exécute une ligne après l'autre. On a le résultat à la suite l'un de l'autre.

```
console.log("premiere ligne execute");  
console.log("seconde ligne execute apres la premiere ligne");  
console.log("troisieme ligne execute apres la deuxieme ligne");
```

Un code asynchrone, ne va pas faire exactement de retour à chaque ligne mais plus tard. Une fonction qui illustre bien ce propos c'est la fonction ; setTimeout()

setTimeout prend en premier argument une fonction puis un temps à attendre avant de return.

```
console.log("premiere ligne execute");  
setTimeout(() => {console.log("deuxieme ligne qui va arrive apres la troisieme"), 100});  
console.log("troisieme ligne execute apres la premiere ligne");
```

```
premiere ligne execute  
troisieme ligne execute apres la premiere ligne  
deuxieme ligne qui va arrive apres la troisieme
```

Pourquoi on met ce genre de chose en place ?

Pour l'expérience utilisateur, le temps d'une requête sur un site peut dépendre de plein de facteurs et entre autres la connexion. Si le site freeze totalement en attendant un retour d'une action de l'utilisateur, c'est très frustrant et pas une super expérience.

Pour la performance, même si on est « single threaded » en javascript, on veut pouvoir réaliser différentes actions. Mais attention, elles ne sont pas faites en même temps mais elles sont faites de manière concurrente.

Promesse the old way

On va dans un premier temps voir une écriture plus « ancienne » de javascript. Mais c'est important pour la suite.

Aussi autre détail important à préciser, on va utiliser setTimeout() pour illustrer nos propos. Mais dans la vraie vie ce n'est pas forcément avec cette fonction, c'est en réalité des fonctions qui vont faire des choses comme fetch(), avoir peut-être un temps de réponse qui est long ou qui dépend d'une action de l'utilisateur.

Une promesse est un objet en javascript qui permet d'avoir le résultat / réponse d'une fonction asynchrone. Ça permet de stocker la valeur qui n'est pas encore disponible.

En terme de syntaxe on fonctionne de la manière suivante (on va décrire le fonctionnement pas à pas) ;

const variable = new Promise(<constructeur>) ;

On doit utiliser le mot new car il s'agit d'un objet.

Maintenant c'est là que les choses se compliquent avec la partie **<constructeur>**

const variable = new Promise((resolve, reject) => {}) ;

Le constructeur d'une promesse prend une fonction avec 2 arguments ; **resolve** et **reject**.

resolve et reject permettent de bien faire passer l'état de la promesse à soit **fulfilled** (dans le cas de **resolve**, un succès) ou **rejected** (dans le cas de **reject** qui est l'échec).

Cette fonction est une arrow function comme on le voit. A l'intérieur du corps de cette fonction on va avoir au lieu d'un return l'utilisation de resolve() et reject(). Ça fonctionne tout comme un return mais ça va nous servir plus tard dans notre code d'agir en fonction du succès ou de l'échec de l'action que l'on souhaitait faire. Return arrête l'exécution de la fonction alors que resolve va juste changer l'état de la promesse.

```
const testPromise = new Promise((resolve, reject) => {
    setTimeout(() => {resolve("my test de promesse");}, 1000);
})

function main() {
    console.log("first line");
    testPromise.then(() => {console.log("yes");});
    console.log("third line");
}

main();
```

Pour l'illustration j'ai utilisé setTimeout() à l'intérieur de notre promesse.

L'output ;

```
~/JS$ nodejs main.js
first line
third line
yes
```

Ici dans ce test on voit que j'ai utilisé testPromise.then(). La méthode **.then()** prend aussi en argument une fonction pour faire l'opération que l'on souhaite. Mais cela n'est enclenché uniquement quand on fait un resolve.

Si on gère aussi l'échec avec reject(), on va devoir utiliser **.catch()** à la place.

The New Way

Maintenant qu'on voit comment ça marche il faut faire la nouvelle méthode.

C'est plus l'esprit syntaxique qu'autre chose qu'on va retrouver ici.

On ne va plus utiliser `.then()` et `.catch()` et « `new Promise()` ». On va utiliser des choses un peu plus jolie et court.

Au lieu de `.then()` on va utiliser **await**.

Au lieu de `.catch()` on va faire une structure **try et catch** classique.

Au lieu de faire un **return new Promise()** etc... on va spécifier **async**. Async va return une nouvelle promesse.

Un exemple ici ;

```
try {
  const message = await promise;
  console.log(`Resolved with ${message}`);
} catch (error) {
  console.log(`Rejected with ${error}`);
}
```

Behind The Magic

La question qui se pose ; comment ça marche si on est single threaded ? Comment Javascript gère tout ça ?

C'est l'environnement de javascript qui fait le gros du travail grâce à « l'event loop ».

Pour ça en arrière-plan on a 2 structure de données ; une stack et une queue.

On connaît la « call stack » dans les autres langages de programmation, ici c'est le même concept. On va à chaque fois qu'on appelle une fonction la mettre dans notre call stack (penser à GDB quand on a une fonction qui appelle une fonction qui appelle une fonction, ou alors du code en assembleur d'un code comme décrit).

A chaque fois qu'on va avoir une fonction asynchrone, on va la mettre dans la « task queue ».

L'event loop c'est ce qui gère les 2 structures de données. C'est assez simple en terme de fonctionnement.

L'event loop va juste mettre des choses sur la stack, une fois que celle-ci est vide on va libérer la task queue. Voilà.....

En réalité, il existe une seconde queue, micro task queue, qui est traitée avant la queue classique pour différentes tâches. Mais pour nous, pour ce document qui n'est qu'un guide de survie, ce document qui n'est que un résumé / prise de note d'un cours.....on s'en fout.

Le REPO de la honte

📖 README 🔗 Contributing 📄 WTFPL license



This project was created by a **developer from Ukraine**.
Russia invaded Ukraine, killing tens of thousands of civilians and displacing millions more.
It's a genocide. Please help us defend freedom, democracy and Ukraine's right to exist.

Help Ukraine Now →

What the f*ck JavaScript?

License WTFPL 2.0 npm v1.22.8 support patreon support buymeacoffee

A list of funny and tricky JavaScript examples

JavaScript is a great language. It has a simple syntax, large ecosystem and, what is most important, a great community.

At the same time, we all know that JavaScript is quite a funny language with tricky parts. Some of them can quickly turn our everyday job into hell, and some of them can make us laugh out loud.

The original idea for WTFJS belongs to [Brian Leroux](#). This list is highly inspired by his talk ["WTFJS" at dotJS 2012](#):



LINK ; <https://github.com/denysdovhan/wtfjs>

*l'auteur de ce document ne considère pas que JS et Typescript sont 2 langages différents comme une partie de la communauté que l'auteur appelle « *les gens du web* » (en reference à la blague historique « peuple de la mer » / « sea people » de l'Egypte ancienne).

L'auteur de ce texte a un avis particulier sur la relation entre les frameworks et javascript, il a exprimé à plusieurs reprises publiquement « *c'est la même merde, arrêtez de faire vos intéressant* » lors de ce qu'on appelle « l'incident de Shibuya » du 31 octobre 2018 ([link](#)).

L'auteur de ce document a refusé de commenter s'il était affilié ou non au clan Zen'in.